

■ KrishiMitra (■■■■■ ■■■■■■) — Complete Technical Q&A & Tech Stack Guide for SIH Hackathon

Repository: <https://github.com/Tushar-Vashistha/KrishiMitra> (<https://github.com/Tushar-Vashistha/KrishiMitra>)

Live Production Frontend: <https://kmitra-in.vercel.app> (<https://kmitra-in.vercel.app>)

Live Production Backend API: <https://krishimitra-backend-rbzu.onrender.com>
(<https://krishimitra-backend-rbzu.onrender.com>)

Interactive API Documentation (Swagger): <https://krishimitra-backend-rbzu.onrender.com/api/docs>
(<https://krishimitra-backend-rbzu.onrender.com/api/docs>)

■ Team Presentation Role Matrix (6 Members)

Member	Role	Primary Focus Area	SIH Viva Q&A Responsibility
Member 1	Main Presenter / Pitch Lead	Presentation, Vision, Live Demo Workflow	Problem Statement, Business Value, e-NAM Comparison
Member 2	Frontend Lead	React 19, Vite, Tailwind CSS v4, Auth State	SPA Architecture, Component Lifecycle, UI Optimization
Member 3	GIS & Localization Specialist	Leaflet Maps, i18next Translation, Haversine Math	Map Tile Rendering, i18n Context, GPS Proximity Formula
Member 4	Backend & Security Engineer	Express REST API, JWT, Bcrypt, Rate Limiter, Helmet	API Endpoints, JWT Security, DDoS Defense, Swagger
Member 5	Database & DevOps Specialist	Supabase PostgreSQL, Prisma ORM, SHA-256 Hashing	22 Prisma Models, ACID Transactions, Data Privacy, Deployment
Member 6	AI & Core Algorithms Engineer	Gemini 1.5 Flash, Offline Fallback Bot, Trust Score	LLM System Prompts, Fallback Intent Engine, Queue State Machine

■ PART 1: Frontend Tech Stack Questions & Simple Explanations

Q1: What is React 19 and why did you use it instead of older React 18 or Create React App (CRA)?

• Simple Answer:

Create React App (CRA) is now outdated and slow. We used **React 19** with **Vite** because:

- React 19 introduces automatic performance optimizations, better concurrent rendering, and cleaner hook hooks ([useActionState](#), [useFormStatus](#)).
- It gives us a faster development experience and smaller JavaScript production bundles, making the site load fast even on slow 3G mobile networks.

Q2: What is Vite and why is it better than Webpack?

- **Simple Answer:**

Vite is a modern build tool for frontend apps. Standard Webpack bundles the entire app code before running, which takes 30–60 seconds. Vite uses **Native ES Modules (ESM)** in the browser, so the dev server starts instantly ($<300\text{ms}>$) with **Hot Module Replacement (HMR)**—meaning code changes reflect instantly in the browser without losing UI state.

Q3: What is Tailwind CSS v4 and how is it configured in KrishiMitra?

- **Simple Answer:**

Tailwind CSS is a utility-first CSS framework. Instead of writing custom `.css` files with class names like `.button-green`, we write utility classes directly in JSX like `className="bg-emerald-600 hover:bg-emerald-700 text-white rounded-lg p-4"`.

- **Configuration:** In Tailwind v4, we don't need a heavy `tailwind.config.js`. It is imported directly in `src/index.css` via `@import "tailwindcss";`, providing high build speed and low CSS file size.
-

Q4: How does Client-Side Routing work with React Router DOM v7? What are Protected Routes?

- **Simple Answer:**

In a Single Page Application (SPA), browser page reloads are prevented. `react-router-dom` v7 intercepts URL changes and renders the corresponding component (`<Route path="/farmer/dashboard" element={<FarmerDashboard />} />`).

- **Protected Routes:** We wrapped private pages (like Farmer Dashboard or Centre Live Queue) in custom route guards. If a user tries to open `/farmer/dashboard` without a valid JWT token in `localStorage`, they are automatically redirected to `/login`.
-

Q5: How does Multi-Lingual Translation (i18next) work without reloading the page?

- **Simple Answer:**

We stored translation JSON files for English (`en`) and Hindi (`hi`) in `src/i18n/`. We wrap components with the `useTranslation()` hook: `{t('welcome_message')}`.

When a farmer clicks the "■■■■■ / English" toggle button, `i18n.changeLanguage('hi')` updates the React state context. React immediately re-renders all text labels into Hindi instantly without refreshing the browser window.

Q6: How do Leaflet and React-Leaflet render interactive maps?

- **Simple Answer:**

`react-leaflet` wraps the open-source Leaflet JS engine inside React components (`<MapContainer>`, `<TileLayer>`, `<Marker>`, `<Popup>`).

1. It fetches free map tiles from OpenStreetMap.
 2. It gets the farmer's current GPS location via `navigator.geolocation`.
 3. It fetches nearby procurement centres from our backend API and plots interactive markers showing centre name, open/closed status, and distance in km.
-

Q7: What is the `useAuth.jsx` custom hook and how does authentication persistence work?

- **Simple Answer:**

`useAuth` is a custom React Hook that manages user login state globally. When a user logs in, the backend returns a user object and JWT Access Token, which `useAuth` saves in the browser's `localStorage` (`krishimitra_user`). When the user refreshes or reopens the browser, `useAuth` automatically reads `localStorage` so the farmer stays logged in smoothly.

Q8: What is the `api.js` service abstraction layer?

- **Simple Answer:**

Instead of writing `fetch('http://localhost:8080/api/v1/...')` in every component, we created `src/services/api.js`. It contains helper methods (`api.get`, `api.post`).

- **Automatic Bearer Token Injection:** `api.js` automatically reads the JWT token from `localStorage` and attaches `Authorization: Bearer <token>` header to every API request.
 - **Auto Logout on 401:** If the backend returns `401 Unauthorized` (token expired), `api.js` automatically clears `localStorage` and redirects the user to the login screen.
-

■■ PART 2: Backend Tech Stack Questions & Simple Explanations

Q9: Why Node.js and Express.js for the Backend?

- **Simple Answer:**

Node.js runs JavaScript on the server using an asynchronous, event-driven, non-blocking I/O model. This makes it ideal for handling thousands of simultaneous farmer requests (booking slots, tracking queue tokens) with low memory usage. Express.js is the lightweight web framework used to create clean RESTful routes (`app.get`, `app.post`).

Q10: What is REST API Architecture and why do you use `/api/v1` versioning?

- **Simple Answer:**

REST (Representational State Transfer) is a standard architectural style for web APIs using standard HTTP methods:

- **GET:** Read data (e.g. fetch MSP rates).
 - **POST:** Create data (e.g. book a procurement slot).
 - **PUT/PATCH:** Update data (e.g. update queue token status).
 - **DELETE:** Remove data.
 - **`/api/v1` Versioning:** Adding `/v1` in the URL path ensures backward compatibility. If we release version 2 (`/api/v2`) in the future, old mobile apps or integrations won't crash.
-

Q11: How does JWT (JSON Web Token) Authentication work? What are Access and Refresh Tokens?

- **Simple Answer:**

JWT is a secure, stateless authentication mechanism consisting of three parts separated by dots:

`Header.Payload.Signature`.

1. When a user logs in with valid credentials, the backend generates a signed JWT containing their `userId` and `role`.

2. The client attaches this token in HTTP Headers: `Authorization: Bearer <JWT_TOKEN>`.
 3. The backend verifies the token signature using a secret key (`JWT_SECRET`).
- **Access Token:** Short-lived token (e.g. 1 hour) used for API requests.
 - **Refresh Token:** Long-lived token (e.g. 7 days) stored securely to request a new Access Token without forcing the user to log in again.
-

Q12: How does Bcrypt password hashing work? What are Salt Rounds?

- **Simple Answer:**

Plain-text passwords should NEVER be saved in a database. **Bcrypt** is a cryptographic hashing function based on the Blowfish cipher.

- **Salt Rounds (10 rounds):** Bcrypt generates a random string called a "salt" and combines it with the password, hashing it $2^{10} = 1024$ times. Even if two users have the same password (e.g. `password123`), their stored hashes will look completely different, protecting against rainbow table attacks.
-

Q13: What is Zod Validation and why validate request bodies on the backend?

- **Simple Answer:**

Never trust data coming from the frontend (users can modify API requests using Postman or browser DevTools). **Zod** is a schema validation library in `src/utils/validators.js`. Before processing a slot booking, Zod verifies:

- Is `weight` a positive number?
- Is `mobile` exactly 10 digits?
- Is `slotTime` in valid format?

If validation fails, Zod rejects the request immediately with a clear error message before reaching the database.

Q14: What is Helmet Middleware and what security headers does it add?

- **Simple Answer:**

`helmet()` is an Express security middleware that automatically sets essential HTTP headers to protect against common web vulnerabilities:

- **X-Frame-Options: DENY:** Prevents Clickjacking attacks (stopping others from embedding our app in an `<iframe>`).
 - **X-Content-Type-Options: nosniff:** Stops MIME-type sniffing exploits.
 - Removes **X-Powered-By: Express:** Hides technology details from hackers.
-

Q15: What is CORS and why is it needed between Vercel and Render?

- **Simple Answer:**

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks web pages from making API requests to a different domain.

Since our frontend is hosted on **Vercel** (`kmitra-in.vercel.app`) and backend is hosted on **Render** (`krishimitra-backend.onrender.com`), the browser blocks API calls by default. We configure `cors()` in Express to explicitly allow requests coming from our Vercel frontend URL.

Q16: What is `express-rate-limit` and how does it prevent DDoS and Brute-Force attacks?

- **Simple Answer:**

`express-rate-limit` tracks request rates per IP address.

- **Auth Rate Limit:** Restricts login/OTP attempts to maximum 15 requests per 30 seconds to prevent password brute-forcing.
 - **Global Rate Limit:** Restricts general API requests to maximum 100 requests per 15 minutes per IP to prevent Denial of Service (DDoS) bot traffic.
-

Q17: What is Winston Logger and why log to files (`logs/error.log`)?

- **Simple Answer:**

Standard `console.log()` is inefficient in production. **Winston** is a multi-transport logging library configured in `src/utils/logger.js`. It logs clean, timestamped messages with severity levels (`info`, `warn`, `error`) both to the terminal console and automatically writes critical crashes into `logs/error.log` for debugging.

Q18: What is Swagger UI (`/api/docs`) and how is it generated?

- **Simple Answer:**

Swagger UI provides an interactive web interface where developers and hackathon judges can test all backend APIs directly inside the browser. We write JSDoc comments above API routes, and `swagger-jsdoc` + `swagger-ui-express` parse those comments to serve the documentation live at `/api/docs`.

■■ PART 3: Database & DevOps Questions & Simple Explanations

Q19: Why PostgreSQL and Supabase instead of MongoDB?

- **Simple Answer:**

Crop procurement requires strict relational integrity and financial transaction accuracy (Farmer $\rightarrow$ Booking $\rightarrow$ Queue Token $\rightarrow$ Weight Log $\rightarrow$ Payment Ledger).

- **Relational Database (PostgreSQL):** Supports ACID transactions (Atomicity, Consistency, Isolation, Durability) ensuring data never gets corrupted or lost.
 - **Supabase:** Cloud-hosted managed PostgreSQL with built-in connection pooling (PgBouncer) for high performance.
-

Q20: What is Prisma ORM? How does `schema.prisma` work?

- **Simple Answer:**

Prisma is an Object-Relational Mapper (ORM) for Node.js. Instead of writing complex raw SQL queries like `SELECT * FROM "FarmerProfile" WHERE ...`, Prisma auto-generates a type-safe JavaScript client: `prisma.farmerProfile.findUnique()`.

- **`schema.prisma`:** The single source of truth file defining all 22 database models, fields, types, and primary/foreign key relationships.
-

Q21: What is the difference between `npx prisma migrate dev` and `npx prisma generate`?

- **Simple Answer:**

- `npx prisma migrate dev`: Creates SQL migration files and applies structural schema changes (like adding a new column) directly to the PostgreSQL database.
 - `npx prisma generate`: Reads `schema.prisma` and updates the JavaScript `@prisma/client` library code inside `node_modules` so controllers can call the new fields.
-

Q22: What is the difference between `DATABASE_URL` and `DIRECT_URL` in Supabase?

- **Simple Answer:**

- `DATABASE_URL`: Connects to Supabase PgBouncer **Transaction Connection Pooler** (Port 6543). Used by the live backend server to handle thousands of concurrent queries without exhausting database memory limits.
 - `DIRECT_URL`: Connects directly to the PostgreSQL database engine (Port 5432). Used strictly by Prisma CLI for executing database migrations (`prisma migrate`).
-

Q23: How do you guarantee Data Privacy for Aadhaar and Bank Account details?

- **Simple Answer:**

We follow strict data privacy compliance:

1. **SHA-256 One-Way Hashing**: Raw Aadhaar and Bank Account numbers are never saved in plain text. We calculate a SHA-256 hash (`aadhaarHash`, `accountNumberHash`) saved in DB to check for duplicate accounts without knowing the actual number.
 2. **Data Masking**: API responses and UI components only display masked strings (`maskAadhaar()` returns `XXXX XXXX 1234`), protecting farmers from identity theft.
-

Q24: What are Database Transactions (`prisma.$transaction`) and why are they needed?

- **Simple Answer:**

A database transaction ensures that multiple SQL operations succeed or fail together as a single atomic unit.

- **Example**: When a crop procurement completes, 4 actions must happen together:

1. Token status updates to `COMPLETED`.
2. `WeighingRecord` and `QualityInspection` rows are saved.
3. `Payment` ledger entry is created.
4. Farmer Trust Score increases by \$+10\$.

By wrapping these inside `prisma.$transaction([])`, if any single step fails (e.g. database network glitch), the entire operation rolls back automatically, preventing partial data corruption.

Q25: How are Vercel (Frontend) and Render (Backend) configured for deployment?

- **Simple Answer:**

- **Frontend (Vercel)**: Connected directly to our GitHub repository `Tushar-Vashistha/KrishiMitra`. Whenever code is pushed to `main`, Vercel runs `npm run build` (Vite) and deploys static assets to global CDN edge nodes with `vercel.json` SPA route rewrites.

- **Backend (Render):** Dockerized/Node.js web service running `npm start (node src/server.js)`, connected to environment variables (`DATABASE_URL`, `JWT_SECRET`, `GEMINI_API_KEY`).
-

■ PART 4: Algorithms & Mathematical Questions & Simple Explanations

Q26: How does the Haversine Distance Formula work to find the nearest procurement centre?

- **Simple Answer:**

The Earth is a sphere, so calculating distance between two GPS coordinates using simple flat geometry ($x^2 + y^2$) is inaccurate.

The **Haversine Formula** in `src/utils/helpers.js` calculates the great-circle distance over Earth's curved surface:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta \lambda}{2}\right)}\right)$$

- Input: Farmer GPS (lat_1, lon_1) and Centre GPS (lat_2, lon_2) .
 - Output: Exact driving distance in kilometers (d), returning centres sorted from closest to farthest.
-

Q27: How does the Trust / Credit Score Algorithm (+10 / -25) work?

- **Simple Answer:**

The Trust Score gamifies punctuality and eliminates ghost slot bookings:

- **Initial Baseline:** Every farmer starts with \$100.0\$ points.
 - **On-Time Arrival:** \$+10\$ points awarded when arriving within the booked 1-hour slot.
 - **No-Show Penalty:** \$-25\$ points deducted if the farmer misses their slot without 24h prior cancellation.
 - **Blacklist Enforcement:** If Trust Score drops to 25\$ points, `booking.service.js` blocks prime daytime slots (07:00 - 17:00) and restricts the farmer to an evening penalty slot (17:00 - 20:00), capped at maximum 2 blacklisted bookings per centre per day.
-

Q28: How does the Multi-Counter Digital Queue State Machine work?

- **Simple Answer:**

To prevent queue confusion at mandis, queue tokens progress through strict state transitions:

1. **WAITING:** Slot booked, farmer waiting for their turn.
 2. **CALLED:** Centre operator calls token to a specific counter (e.g. Weighing Counter 2).
 3. **ARRIVED:** Farmer checks in at the designated counter.
 4. **PROCESSING:** Crop weighing and quality lab test in progress.
 5. **COMPLETED:** Inspection passed and transaction logged (or **REJECTED** if quality fails).
 6. **NO_SHOW:** Triggered if farmer doesn't respond when called (deducts 25 Trust points).
-

Q29: What is Tatkaal Emergency Booking and how does it work?

- **Simple Answer:**

If unexpected rain or storm threatens harvested crops, farmers cannot wait days for standard slots. **Tatkaal Booking** allows emergency priority booking by paying a system-configurable priority fee (default ₹50 configured in `SystemSetting`), issuing a priority token flagged `isTatkaal: true`.

■ PART 5: AI & Chatbot Questions & Simple Explanations

Q30: How does the Google Gemini 1.5 Flash AI Assistant work?

- **Simple Answer:**

We integrated `@google/generative-ai` connected to the `gemini-1.5-flash` model.

- **System Instruction (`KRISHIMITRA_SYSTEM_PROMPT`):** We pass a detailed system prompt in `chatbot.service.js` specifying MSP rates (Wheat ₹2,275/Qtl, Paddy ₹2,183/Qtl), slot policies, trust rules, and multi-lingual tone. This prevents the AI from giving incorrect or irrelevant information.
-

Q31: What is the Bilingual Offline Fallback Intent Matcher Engine?

- **Simple Answer:**

If internet connection drops or Gemini API quota is exhausted during a hackathon demo, our custom **Fallback Intent Engine** in `chatbot.service.js` takes over automatically. It scans the farmer's query in Hindi or English, scores matching keywords against 10+ pre-configured response templates (MSP rates, slot booking, trust score, payments), and returns an accurate answer with zero downtime.

■ PART 6: General SIH Hackathon & Jury Trick Questions (Simple Explanations)

Q32: Why a Web App instead of a Mobile App (Flutter/React Native)?

- **Simple Answer:**

Farmers have diverse mobile devices with limited storage space. A Progressive Web App (PWA) built with React 19 requires **zero installation**, opens instantly in any mobile browser, supports home-screen shortcut installation, and uses $<2\text{MB}$ data.

Q33: What if a small farmer has low 2G internet or no smartphone?

- **Simple Answer:**

1. **Low Internet:** Our frontend static assets are cached via Vercel CDN, and payload bodies are restricted to $<10\text{kb}$.
 2. **No Smartphone:** Mandi staff and local Common Service Centres (CSCs) have access to the `CENTRE_STAFF` portal to register farmers, book slots, and print physical paper queue tokens on their behalf.
-

Q34: What if 1,000 farmers arrive at the same procurement centre at once?

- **Simple Answer:**

This cannot happen because our slot booking engine enforces **Slot Capacity Limits** ([capacity](#) in [SlotConfig](#)). A centre handling 100 Quintals/day will only issue 10 slot tokens per hour. If a slot time is full, farmers must select a different time slot or nearby centre.

Q35: How do you prevent traders or middlemen from selling commercial grain under a farmer's name?

- **Simple Answer:**

During registration, land Khasra details and land area (in acres) are verified against crop yield benchmarks ([estimatedYield](#) in [FarmerCropRegistration](#)). If a farmer owns 2 acres of land, the system caps their maximum procurement booking to their estimated yield limit (e.g. 40 Quintals), blocking traders from selling excess commercial grain under their identity.